颜欣杰

18779910339 | yanxinjie06@163.com
20 岁 男 AI应用开发工程师 https://github.com/jiezhouziheng



教育背景

南京大学 软件工程（智能化软件）（本科） 2024.09 - 2028.06

项目经历

Kea Agent | 本地终端编码智能体

2026.08 - 至今

TypeScript, Bun, React, OpenTUI

- 设计分层终端编码智能体架构，拆分 AI Protocol、Agent Harness、Coding Agent 和 CLI/TUI UI 层，降低模型协议、执行逻辑与交互界面的耦合。
- 统一 OpenAI、Anthropic、Gemini 三种模型协议，支持流式文本、思考内容、Tool Call、Token 用量、模型路由和兼容 OpenAI 协议的自定义 Base URL。
- 实现多轮 Agent Loop，支持工具调用、工具结果回填、最大轮数限制、AbortSignal 取消和异常终止处理。
- 基于 TypeBox 和 Tool Registry 实现工具参数校验、权限 Hook、超时控制、错误结果封装和结构化执行元数据。
- 实现 Project 级权限治理，支持外部目录确认、Allow once、Always allow、Deny、高风险 Bash 命令拦截和 fail-closed 行为。
- 设计基于 JSONL 的树形 Session 存储，支持会话恢复、模型选择记录、延迟持久化和 Session Fork。
- 构建 CLI 与 OpenTUI React 双界面，共享命令解析、Runtime、Session、模型切换、权限交互和退出清理流程。
- 完成 Bun 版本锁定、TypeScript 严格类型检查、Windows/Ubuntu CI、dist 产物校验和全局安装验证。

LifeGraph | 个人生活图谱应用 AI 助手与 Agent 模块开发

2026.04 - 2026.06

Expo/React Native, TypeScript, SQLite/Drizzle, FastAPI, OpenAI-compatible LLM API

- 负责AI助手模块的端到端开发，包括移动端对话界面、会话状态管理、API Client，以及 FastAPI LLM Gateway 的接口协议和错误处理。并升级助手为任务型Agent，设计从自然语言输入到结果输出的多阶段执行流程。
- 基于 Function Calling 扩展人物、事件、关系和统计类工具，使用户可以通过自然语言完成图谱查询及数据维护。
- 设计本地优先的 Agent 执行边界：LLM 只负责意图理解、任务规划和工具调用，不直接访问用户数据库；由移动端本地 Agent 调用领域服务和 Repository，保护用户数据隐私。
- 针对写操作增加重复检测、字段校验、歧义澄清和用户确认机制；对人物、关系、事件等多表操作使用事务控制，保证多步骤任务的一致性并支持失败回滚。
- 建立统一的任务计划、工具参数、执行结果和错误码协议，记录原始指令、计划内容、用户确认、工具调用、执行耗时及最终结果，便于失败恢复和问题定位。
- 使用 Jest 和 Pytest 覆盖 Agent 多轮调用、工具参数校验、未知工具、最大执行轮次、网络异常、LLM Provider 异常、计划执行失败及事务回滚等场景。

专业技能

- **Agent/LLM 应用开发：**熟悉多轮 Agent Loop、任务规划、Function Calling/Tool Calling、工具注册与路由、结构化任务计划、参数校验、工具结果回填、流式响应及模型路由；具备 OpenAI、Anthropic、Gemini 协议适配和 OpenAI-compatible API 开发经验。
- **Agent 工程与安全治理：**掌握 Tool Registry、权限 Hook、用户确认、Allow once/Always allow/Deny、fail-closed、高风险命令拦截；具备最大执行轮次、超时控制、AbortSignal 取消、异常处理和结构化执行记录设计经验。
- **Agent 架构与可靠性：**熟悉“LLM 规划 → 本地 Agent → 领域服务/Repository → 数据库”的本地优先架构；具备任务计划、工具参数、执行结果和错误码协议设计，以及重复检测、歧义澄清、事务回滚、会话恢复与 Session Fork 实践。
- **编程语言与框架：**TypeScript/JavaScript、Python、SQL；Bun、FastAPI、React、React Native/Expo、OpenTUI；SQLite、Drizzle ORM、JSONL 持久化。
- **工程化与测试：**TypeBox Schema 校验、Jest、Pytest、TypeScript Strict Mode、异步与流式处理、跨平台 CI、构建产物校验和全局安装验证。